ALGORITHME

PLAN DU COURS

CHAPITRE 1 : LES INSTRUCTIONS DE BASES

CHAPITRE 2 : LES STRUCTURES CONDITIONNELLES

CHAPITRE 3 : LES STRUCTURES REPETITIVES

CHAPITRE 4 : TABLEAUX

CHAPITRE 5 : FONCTIONS ET PROCEDURES

CHAPITRE 6 : TABLEAUX A DEUX DIMENSIONS

CHAPITRE 7 : RECURSIVITE

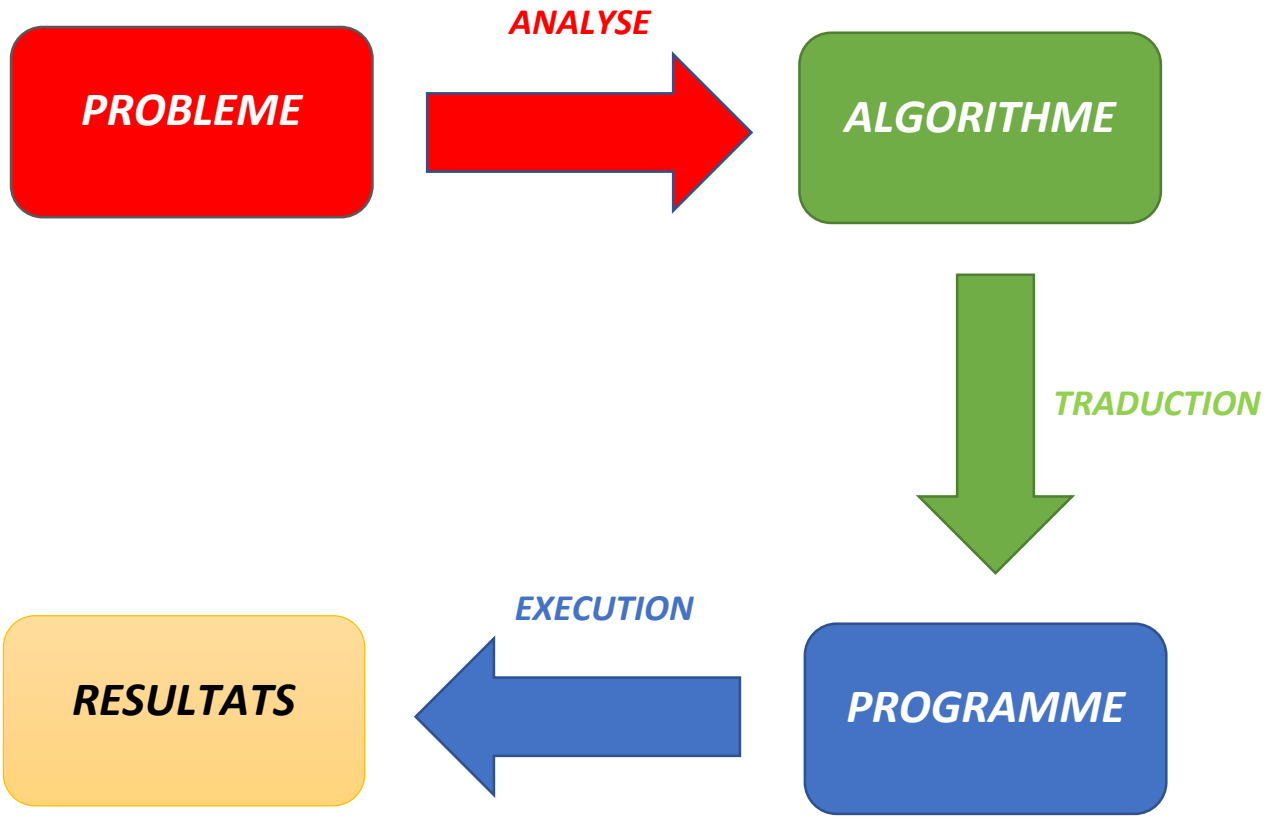
CHAPITRE 8 : COMPLEXITE

CHAPITRE 1 : LES INSTRUCTIONS DE BASES

I) DEFINITION D'UN ALGORITHME

Un algorithme est une suite d'actions ou d'instructions élémentaires qui doivent être exécutées dans un ordre bien déterminées pour résoudre un problème (ou réaliser un travail).

La résolution informatique d'un tel problème comporte les phases suivantes :



II) TYPE DE DONNEE

Les données sont des informations nécessaires au déroulement d'un algorithme. On distingue deux catégories :

- *CONSTANTE*
- *VARIABLE*

a) CONSTANTE

Une donnée fixe qui ne change pas durant l'exécution de l'algorithme. Il est caractérisé par son nom et sa valeur.

Exemple :

Nom (Sidi ; Rama ; Ansou42 ‘’ @ ‘’ etc…) : Chaine de Caractère

Valeur (120 ; 25 ; 1 ; 45 ; 99 etc…) : Entier

b) VARIABLE

Une donnée dont le contenu peut être modifiée par une action durant l’exécution de l’algorithme. Une donnée qui change.

Exemple :

Nom (Moussa ; Fatou ; Awa etc…) : Chaine de Caractère

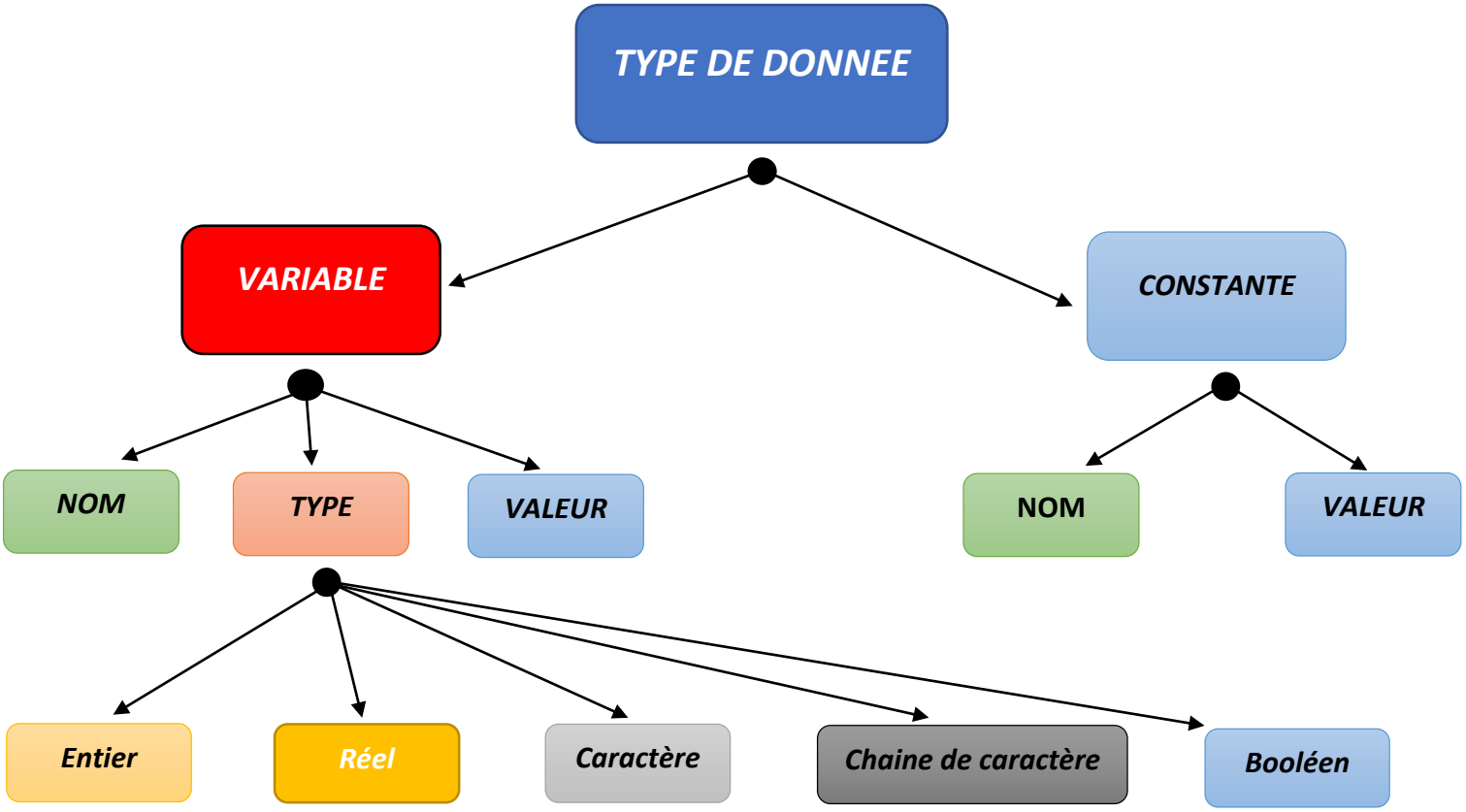
Numéro (1 ; 12 ; 86 ; 024 etc…) : Entier

Sexe (M ; F ‘’f’’ etc…) : Caractère

Note (10,45 ; 12,75 ; 14,02 ; 15,25 etc…) : Réel

Logique (vrai ; faux etc…) : Booléen

- ***LE SCHEMA***



- ***TABLEAU***

TYPE DE DONNEE		EXEMPLES
NUMERIQUE	ENTIER	-375 ; 49 ; 2019 ; -325 ; 10
	REEL	-451 ; 3 ; $4,5.10^2$; 14,10
ALPHA NUMERIQUE	CARACTERE	''A'' ; ''2'' ; ''$'' ; ''n''
	CHAINE DE CARACTERE	''ASSANE'' ; ''MOUSSA 15'' ; ''35012'' ; ''bonjour''
BOOLEEN		Vrai ; faux

Exercice d'application

Donnez les types de donne suivante :

Donnée	Type
''Bienvenue au Sénégal''	
-300	
''8''	
256.5	
'' @ ''	
''vrai''	
Faux	

III) SYNTAXE DE DECLARATION

La déclaration permet d'informer l'ordinateur l'existence d'une donnée. C'est-à-dire demande l'ordinateur la permission de réserver une espace de la mémoire ou l'on peut stoker et récupéré l'information.

*Syntaxe de Déclaration d'une variable et d'une constante :

- Constante Nom_constante = Valeur
- Variable Nom_variable : Type

Exemple :

➢ Constante :

Constante Pi = 3,14

Constante nbr_Mois = 12

Constante nom_Ecole = ''Lycée Léopold Sedar Senghor''

➢ Variable

Variable num_Etudiant : Entier

Variable note : Réel

Variable prénom : Chaine de Caractère

Variable valide : Booléen

Variable operateur : Caractère

IV) *AFFECTATION*

L'affectation est une opération qui consiste à attribuer à une variable.

- Soit une valeur particulière
- Soit une valeur dans une autre variable
- Soit une valeur calculée à l'aide d'opérateur arithmétique

Elle est représentée par une flèche orientée à gauche ←

➢ Syntaxe d'Affectation

Nom_variable ← Valeur

Exemple :

A ← 2 La variable A reçoit la valeur 2

B ← A La variable B reçoit le contenu de A

C ← A + B La variable C reçoit le résultat de A + B

Nom ← ‘’Ali’’ La variable nom reçoit la valeur Ali

Exercice : Affectation Et Déclaration

Exo 1 :

Soient les variables A, B et C tels que :

- A est un Entier
- B est une Chaine de Caractère
- C est de type Logique

	A ← 1		B ← ‘’Maroc’’
	A ← ‘’Karl Max’’		C ← 10
	B ← 3		C ← 2<5
	B ← A		C ← 1<-22

1) Comment on déclare les variables A, B et C dans ce cas ?
2) Cochez ceux qui sont juste :

Exo 2 :

	Variables			
Instructions	A	B	C	D
B ← 2				
C ← B + 10				
A ← 4				

D ← A				
B ← B * D				
C ← B + 5				
A ← 10 + 4 + C				
C ← A + B + D				

V) ECRIRE ET LIRE

a) Ecrire (opération de sortie)

L'instruction écrire permet d'afficher la valeur d'une expression sur un périphérique de sortie (écran).

Une expression peut être :

- Un nombre
- Une numérique
- Un résultat d'une opération entre plusieurs variables
- Une chaine de caractère, dans ce cas, il est nécessaire de mettre la chaine de caractère entre deux guillemets.

➢ Syntaxe d'écriture :

1. Ecrire (variable)
2. Ecrire (''Message'')
3. Ecrire (''Message'', variable)

Exemples :

Ecrire (A) ➡ Signifie affiché sur l'écran le contenu de la variable A

Ecrire (''Dakar'') ➡ signifie affiché sur l'écran le message suivant : Dakar

Ecrire (''A='', A) ➡ signifie affiché sur l'écran le message suivant : A=, plus le contenu de la variable A.

b) Lire (opération d'entrée)

L'instruction Lire permet de demander à l'utilisateur de fournir des informations. Chaque information donnée par l'utilisateur est stockée dans une variable (attention au type !).

➢ Syntaxe d'écriture :

- Lire (variable1)

- Lire (variable1, variable2…)

Exemples :

Lire (x) ⟶ Pour demander la valeur de x

Lire (nom, prénom, âge) ⟶ Pour demander le nom, prénom et âge

Remarque :

Lors de l'exécution de l'instruction lire la machine attend que l'utilisateur lui fournisse une valeur afin de pouvoir continuer à exécuter l'algorithme.

Exercice : Ecrire

Soient X, Y et Z trois variables :

X ⟵ 150

Y ⟵ 100

Z ⟵ '' DH ''

Qu'affichent les instructions suivantes :

Instruction	Résultat
Ecrire (''Hello Word !'')	
Ecrire (X)	
Ecrire (''Z'')	
Ecrire (X * 2)	
Ecrire (''prix :'', X + Y, Z)	

Exercice : Ecrire Et Lire

Nous voulons écrire un algorithme qui calcule l'aire d'un cercle.

1) Donner les instructions de déclaration.
2) Donner les instructions qui demandent à l'utilisateur de taper les valeurs des données.
3) Donner les instructions de traitement.
4) Donner les instructions qui permettent d'afficher le résultat.

VI) LA STRUCTURE D'UN ALGORITHME

Algorithme Nom_Algorithme ← **Entête de l'Algorithme**

- Variable

  Nom_variable : type

- Constante

  Nom_constante = valeur

} **Bloc de Déclaration de donnée**

Début

_

_

_

_

_

Fin

} Bloc D'instruction

Exercice d'application

Ecrire un algorithme qui demande à l'utilisateur de taper la quantité de produits vendus, le prix et qui affecte le chiffre d'affaires de l'entreprise.

VII) EXPRESSION ARITHMETIQUES

Une expression arithmétique est formée par des combinaisons d'objets (entier, réel) et des opérateurs arithmétiques.

Une expression arithmétique donne un résultat numérique dont le type est entier ou réel.

Les opérateurs arithmétiques usuels sont :

OPERATEURS	SIGNIFICATION
+ ; -	Addition ; Soustraction
* ; /	Multiplication ; division
Div	Division entière
mod	Reste de la division
^	Puissance

Exercice : Expressions Arithmétiques

Donner les valeurs des variables après l'exécution de chaque instruction :

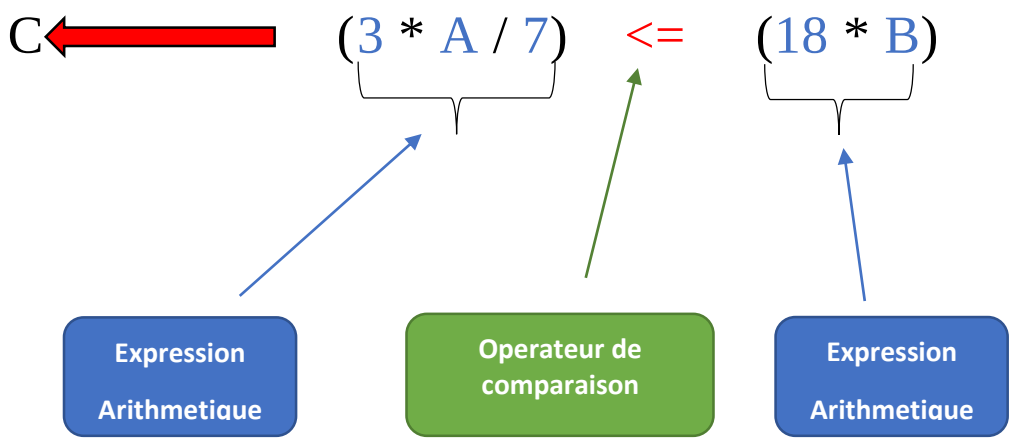
INSTRUCTIONS	RESULTATS
A ← 4 ^ 2	
B ← 8 * 5	
N ← A + B	
P ← N – 20	
R ← B / 3	
X ← B mod 3	
Y ← B div 3	

VIII) EXPRESSION DE COMPARAISON

Elle est une comparaison entre deux expressions arithmétiques. Une expression de comparaison donne un résultat booléen (vrai ou faux).

Les opérateurs de comparaison usuels sont : >, =, <, >=, <=, <>

Exemple :



Exercice : Expression de Comparaison

Donner les valeurs des v variables après l'exécution de chaque instruction :

INSTRUCTION	RESULTAT
F ← 3 < 5	
G ← 2 <> 4	
H ← 1 > 8	
X ← 4 + 5	
K ← (X mod 3) = (X ^ 8)	
J ← (X * 10) >= 90	
M ← (X * X + X) > (X ^ 3)	

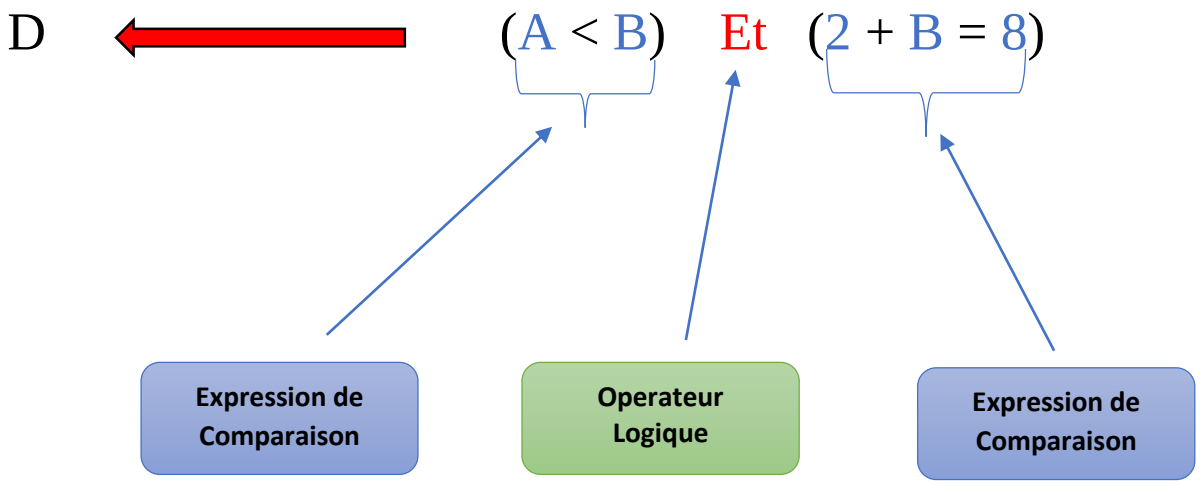
IX) EXPRESSIONS LOGIQUES

Une expression logique est la composée d'expressions de comparaisons par les opérateurs logiques. Une expression logique donne un résultat booléen (vrai ou faux).

Les opérateurs logiques usuels sont : Et, Ou, Non

- Et : les deux valeurs doivent être Vrai pour que le résultat de cette expression soit Vrai.
- Ou : au minimum une des valeurs doit être Vrai pour que le résultat de cette expression soit Vrai.
- Non : c'est l'inverse du résultat.

Exemple :



- Expressions Logiques

Soit A et B deux variables booléennes. Le tableau suivant illustre les différentes valeurs de vérité que l'on obtient en combinant les valeurs de A et B à l'aide des opérateurs logiques.

A	B	A Et B	A Ou B	Non A
Faux	Faux	Faux	Faux	Vrai
Faux	Vrai	Faux	Vrai	Vrai
Vrai	Faux	Faux	Vrai	Faux
Vrai	Vrai	Vrai	Vrai	Faux

Exemple :

INSTRUCTIONS	RESULTAT
A ← 5	
B ← 6	
C ← (A < B) Et (3 > B)	
D ← (A = 5) Ou (B > 10)	
E ← Non (C)	
F ← C Ou (E Et D)	
G ← (Non (E) Et F) Ou (C Et D)	

Exercice : Expressions logiques

Soit A une variable logique, quelle sera la valeur de A dans chacun des cas suivants :

INSTRUCTION	RESULTAT
A ← Non (3 = 5)	
A ← (2 <> 2) Ou (3 > 4)	
A ← ((30 / 3) = 10) Et (3 > 30)	
A ← ((2 > 4) Et (8 <= 8)) Ou (3<> 4)	
A ← (40 >= 30) Et ((10 ^ 2) <> 50)	
A ← Non ((82 mod 8) / 2 = 1)	

X) LES COMMENTAIRES

- Pour accentuer la lisibilité de l'algorithme et faciliter sa compréhension par les humains, on peut introduire des commentaires, qui ne sont pas des instructions destinées à la machine mais des indications données au lecteur de l'algorithme.
- Les commentaires en algorithme sont encadrés par les symboles %.

➢ Syntaxe :

% Commentaire %

Exemple :

% Déclaration de la variable pays %

Variable pays : Chaine de Caractères

% Affectation de la valeur Sénégal à la variable pays %

Pays ← ''Sénégal''

CHAPITRE 2 : LES STRUCTURES CONDITIONNELLES

I) LA STRUCTURE CONDITIONNELLE

La structure conditionnelle est une structure dont les instructions sont exécutées selon les réponses des conditions.

a) Structures conditionnelles - Structure Simple (un choix)

- Syntaxe :

```
Si Condition Alors
        Instructions
Fin Si
```

Si la condition vaut vrai alors le bloc d'instructions sera exécuté, si non il sera ignoré.

Exercice : Structure Si… Fin Si

Ecrire un algorithme qui permet de calculer le maximum de deux nombres réels saisies par l'utilisateur.

```
Algorithme Maximum
Variables
        A , B , Max : Réel
Début
        Ecrire (''Entrer les valeurs de A et de B :'')
        Lire (A , B )
        Max ← A
        Si Max < B Alors
                Max ← B
        Fin si
        Ecrire (''Le maximum est égal a :'', Max)
```

Fin

b) Structures Conditionnelles – Structure Alternative (a deux choix)

- Syntaxe :

```
Si Condition Alors
        Instructions1
Sinon
        Instructions2
Fin Si
```

Si la condition est vraie alors le bloc d'instruction1 sera exécutée, et le bloc d'instructions2 sera ignoré. Sinon le bloc d'instructions2 sera exécuté et le bloc d'instructions1 sera ignoré.

Exercice : Structure Si…..Alors…….Sinon……Fin Si

```
Algorithme
Variables
    N : Entier
Début
    Ecrire (''Programme qui détermine la nature d'un nombre :'')
    Ecrire (''veuillez entrer un Nombre :'')
    Lire (  )
    Si        Alors
        Ecrire (''Ce nombre est positif '')
    Sinon
        Ecrire ('' Ce nombre est négatif '')
    Fin Si
Fin
```

c) Structure Conditionnelles – Structure Alternative imbriquée (multiple choix)

- Syntaxe :

```
Si Condition Alors
        Instructions1
Sinon
        Si Condition2  Alors
                Instructions2
        Sinon
                Instructions3
        Fin Si
Fin Si
```

Exercice : Structure Imbriquée

Ecrire un algorithme qui permet de lire à valeur de la température de l'eau et 'afficher son état :

- GLACE si la température est inférieure 0
- LIQUIDE si la température est strictement supérieure a 0 et strictement inferieure a 100
- VAPEUR si la température est supérieure à 100.

```
Algorithme Etat_Eau
Variables
    T : réel
Début
    Ecrire (''Veuiller entrer la température :'')
    Lire (T)
    Si T <= 0 Alors
        Ecrire ('' L'état de l'eau est GLACE'')
    Sinon
        Si T >= 100 Alors
            Ecrire (''L'état de l'eau est VAPEUR'')
        Sinon
            Ecrire ('' L'état de l'eau est LIQUIDE'')
        Fin Si
    Fin Si
Fin
```

D) Structure Conditionnelles – Structure à Choix Multiples Cas

Lorsque l'imbrication devient importante, l'utilisation de la structure a multiple choix devient nécessaire.

- Syntaxe :

```
Cas Variable Vaut
    Valeur1 : Instruction 1
    Valeur2 : Instruction 2
      …       :       …
    Valeurn : Instruction n
    Sinon : Autre instruction
Fin Cas
```

Exercice : Structure à Choix Multiple

Ecrire un algorithme qui permet de demander à l'utilisateur de saisir un entier entre 1 et 7 au clavier et afficher le nom du jour correspondant.

```
Algorithme Jour_Semaine
Variables
    N : Entier
Debut
    Ecrire (‘’ Veuiller entrer un nombre entre 1 et 7 :’’)
    Lire (n)
    Cas n Vaut
        1 : Ecrire (‘’Lundi’’)
        2 : Ecrire (‘’Mardi’’)
        3 : Ecrire (‘’Mercredi’’)
        .. : …      (‘’ …..’’)
        7 : Ecrire (‘’Dimanche’’)
        Sinon : Ecrire (‘’Le nombre est incorrect’’)
    Fin Cas
Fin
```

CHAPITRE 3 : LES STRUTURE REPETITIVES

La structure répétitive (boucle) permet d'exécuter plusieurs fois une séquence d'instructions.

Dans une boucle, le nombre de répétition peut être connu, fixe à l'avance, comme il peut dépendre d'une condition permettant l'arrêt et la sortie de cette boucle.

III) La Boucle POUR

Cette boucle permet d'exécuter une séquence d'instructions un nombre de fois connu fixe a l'avance. Elle utilise une variable (indice) de contrôle d'itérations caractérisées par :

- Sa valeur initiale
- Sa valeur finale
- Son pas de variation.

- Syntaxe :

```
Pour I ←——— N à M pas 1 Faire
    Instructions
Fin Pour
```

I : Variable
N : Valeur Initiale
M : Valeur Final
Le pas de variation égale à 1

Exemple :

```
Algorithme Table_7
Variables
        M , I : Entier
Début
        Pour I ←—— 0 à 10 pas 1 Faire
                M ←—— I * 7
                Ecrire ( I , '' * 7 = '' ,M)
        Fin Pour
Fin
```

II) La Boucle Tant Que

Cette boucle permet de répéter un bloc d'instructions tant qu'une condition est vraie.

- Syntaxe

```
Tant que Condition Faire
        Instructions
Fin Tant que
```

Remarque

La vérification de la condition s'effectue avant l'exécution des instructions. Celles-ci peuvent donc ne jamais être exécutées.

Exemple :

```
Algorithme Table_Multiplication
Variables
    M , N , I : Entier
Début
    Ecrire (''Entrer un nombre entre 1 et 10 :'')
    Lire (N)
    Tant que N < 1 ou N > 10 Faire
        Ecrire (''Entrer un nombre entre 1 et 10 :'')
        Lire (N)
    Fin Tant que
    Pour I ←— 0 à 10 pas 1 faire
        M ←— I * N
        Ecrire ( I , '' * '' , N , '' = '' , M )
    Fin Pour
Fin
```

III) La Boucle Répéter… Jusqu'à…

Cette boucle qui permet de répéter un bloc d'instructions jusqu'à ce qu'une condition soit vérifiée.

- Syntaxe :

```
Répéter
        Instructions
Jusqu'à Condition
```

Remarque :

La vérification de la condition s'effectue après l'exécution des instructions. Celles-ci sont donc exécutées au moins une fois.

Exemple :

```
Algorithme Table_Mutiplication
Variables
        M , N , I : Entier
Début
        Répéter
        Ecrire (''Entrer un nombre entre 1 et 10 :'')
        Lire (N)
        Jusqu'à N > 0 et N <= 10
        Pour I ←— 0 à 10 pas 1 Faire
                M ←— I * N
                Ecrire ( I , '' * '' , N , '' = '' , M )
        Fin Pour
Fin
```

CHAPITRE 4: TABLEAU

Un tableau est une suite d'éléments de même type. Il utilise plusieurs cases mémoire a l'aide d'un seul nom. Comme toutes les cases portes le même nom. Elles se différencient par l'indice.

III) Declaration d'un Tableau

- Syntaxe:

Tableau nom_tab (Taille): Type

II) Accéder aux éléments d'un tableau

- Syntaxe d'affectation : nom_tab (indice) ← Valeur
- Syntaxe lecture : Lire (nom_tab (indice))
- Syntaxe d'écriture : Ecrire (nom_tab (indice))

Exemple :

Ecrire un Algorithme qui permet de demander a l'utilisateur de saisir les notes des étudiants (318 étudiants), puis l'algorithme calcule et affiche la moyenne des notes.

```
Algorithme Moyenne_notes
Variables
        Tableau N (318): reel
        M: reel
        I: 24ntire
Debut
        Pour I ← 0 a 317 pas 1 Faire
                Ecrire ("Donner la note de l'étudiant num : ", I + 1 , " :")
                Lire (N (I))
        Fin Pour
        S ← 0
        Pour I ← 0 a 317 pas 1 Faire
                S ← S + N(I)
        Fin Pour
        M ←        S / 318
        Ecrire ("La moyenne des notes est: ", M)
Fin
```

CHAPITRE 5: FONCTION ET PROCEDURE

III) FONCTION

Une fonction est une suite d'instructions regroupées sous un nom ; elle prend en entrée des paramètres (arguments) et retourne un résultat.

Une fonction est écrite séparément du corps de l'algorithme principal et sera appelée par celui-ci lorsque cela sera nécessaire.

- Syntaxe de declaration :

```
Fonction nom_fonction (agr1 : type arg2 : type, …) : type_retour
Variables
        Variable1 : type
        Variable2 : type
            …
Début
    Instructions
    Retourne resultat_retour
Fin
```

Exemple :

Ecrire un Algorithme qui permet de définir et d'appeler une fonction minimum qui renvoie le plus petit de deux nombres différents.

```
Algorithme Exercices_fonction
    Fonction minimum (x : entier, y : entier) : entier
    Variables
        Min : entier
    Début
        Si x >= y Alors
            Min ←—— y
        Sinon
            Min ←—— x
        Finsi
        Retourne Min
    Fin
Variables
    x , y : 26ntire
Début
    Ecrire (''Veuillez entrer les valeurs de x et y :'')
    Lire (x,y)
    Ecrire (''Le minimum est :'', minimum(x,y)
Fin
```

II) PROCEDURE

Une procédure est une suite d'instructions regroupées sous un nom ; elle prend en entrée des paramètres (arguments) mais qui ne retourne rien.

Une procédure est écrite séparément du corps de l'algorithme principal et sera appelée par celui-ci lorsque cela sera nécessaire.

- Syntaxe de déclaration :

```
Procédure nom_procedure (arg1 : type, arg2 : type,…)
Variables
        Variable1 : type
        Variable2 : type
            …
Début
    Instructions
Fin
```

Exemple :

Ecrire un Algorithme qui permet de définir et d'appeler une procédure minimum qui renvoie le plus petit de deux nombres différents.

```
Algorithme Exercices_Procedure
    Procédure minimum (x : entier, y : entier)
    Variables
        Min : entier
    Début
        Si x >= y Alors
            Min ← y
        Sinon
            Min ← x
        Finsi
        Ecrire (''Le minimum est : '', Min)
    Fin
Variables
    x , y : 27ntire
Début
    Ecrire (''Veuillez entrer les valeurs de x et y :'')
    Lire (x,y)
    Minimum (x, y)
Fin
```

CHAPITRE 6 : TABLEAUX A DEUX DIMENSIONS

Un tableau a deux dimensions équivaut à une matrice. Le premier indice représente le numéro de ligne, le second indice le numéro de colonne.

- Syntaxe de Déclaration :

```
Tableau nom_tab (lignes, colonnes) : Type
```

II) Accéder à un élément d'un tableau a deux dimensions

- Syntaxe d'affectation : nom_tab (num_lig, num_col) ← Valeur

- Syntaxe de Lecture : Lire (nom_tab (num_lig, num_col))

- Syntaxe d'écriture : Ecrire (nom_tab (num_lig, num_col))

III) Afficher, Remplir tous les éléments :

```
Pour i ← 0 a lignes -1 pas 1 Faire
  Pour j ← 0 a colonnes -1 pas 1 Faire
      Ecrire (‘’ L'élément ‘’, i + 1 , j + 1 , ‘’ du tableau Test : ‘’, T (i , j)
      Lire (T (i , j))
  Fin Pour
Fin Pour
```

Exemple :

Ecrire un Algorithme qui permet de demander à l'utilisateur de saisir les notes des étudiants (5 étudiants) dans chaque matière (3matieres), puis l'algorithme calcule et affiche la moyenne de chaque étudiant.

```
Algorithme Moyenne_note
Variables
    Tableau Notes (5 , 3) : réel
    M , S : réel
    I , J : entier
Début
    Pour I ← 0 à 4 pas 1 Faire
        Pour J ← 0 à 2 pas 1 Faire
            Ecrire (''Donner la note de l'étudiant numéro :'', I + 1 ,''dans
            la matière numéro :'', J + 1)
            Lire (Notes (I, J))
        Fin Pour
    Fin Pour
    M ← 0
    S ← 0
    Pour I ← 0 à 4 pas 1 Faire
        Pour J ← 0 à 2 pas 1 Faire
            S ← S + Notes (I, J)
        Fin Pour
        M ← S / 3
        Ecrire (''La moyenne de l'étudiant numéro :'', I +1 , ''est'', M)
        S ← 0
    Fin Pour
Fin
```

CHAPITRE 7 : RECURSIVITE

Une fonction récursive est une fonction qui s'appelle elle-même.

Une fonction récursive est définie par :

- Au moins un cas de base et,
- Au moins un cas récursif (cas général).

Exemple :

Ecrire un algorithme qui demande à l'utilisateur de taper un entier positif n, Ensuite, a l'aide d'une fonction récursive, l'algorithme calcule et affiche tous les termes de la suite de Fibonacci, inferieur ou égaux a n. La suite de Fibonacci est définie comme :

$$\begin{cases} U0 = 0 \\ U1 = 1 \\ Un+2 = Un+1 + Un \end{cases}$$

```
Algorithme Suite_fibonacci
    Fonction fib (n : entier) : entier
        Début
            Si n < 2 Alors
                Retourne n
            Sinon
                Retourne fib (n -1) + fib (n- 2)
            Fin si
        Fin

    Variables
        N : entier
Début
    Ecrire (‘’ Veuillez saisir un nombre ‘’)
    Lire (N)
    Ecrire (‘’Les termes de la suite de Fibonacci inferieur ou égaux a ‘’, N , ‘’
sont : ’’)
    Pour i ←———— 0 à N pas 1 faire
    Ecrire (fib (i))
    Fin Pour
Fin
```

CHAPITRE 8 : COMPLEXITE

La complexité d'un algorithme consiste en l'étude de la quantité de ressources (de temps ou d'espace) nécessaire a l'exécution de cet algorithme.

- La complexité temporelle d'un algorithme quantifie le temps nécessaire a un algorithme pour s'exécuter en fonction de la longueur d'entrée.
- La complexité spatiale d'un algorithme quantifie la quantité d'espace ou de mémoire prise par un algorithme pour s'exécuter en fonction de la longueur d'entrée.

❖ Types de Complexité :

Si le temps d'exécution d'une instruction est de 10 nanosecondes (1/1000000000 de seconde). Le tableau suivant indique le temps d'exécution de chaque type de complexité.

Temps	Type de Complexité	Temps N = 5	Temps N = 10	Temps N =20	Temps N=50	Temps N=250	Temps N=1K	Temps N=10K	Temps N=1M
O (1)	Constante	10 ns	10 ns	10 ns	10 ns	10 ns	10 ns	10 ns	10 ns
O(log(n))	Logarithmique	10 ns	10 ns	10 ns	20 ns	30 ns	30 ms	40 ns	60 ns
O(n)	Linaire	50 ns	100 ns	200 ns	500 ns	2.5 µs	10µs	100 µs	10 ms
O(nlog(n))	Quasi-linaire	50 ns	100 ns	200 ns	501 ns	2.5 µs	10 µs	100 µs	100.5ms
O(n^2)	Quadratique	250 ns	1 µs	4 µs	25 µs	625 µs	10 ms	1 sec.	2.8 H
2^poly(n)	Exponentielle	320 ns	10 µs	10 ms	130 jours	10^59 Ans	…	…	…
O (n !)	Factorielle	1.2 µs	36 ms	770 ans	10^48 Ans	…	…	…	…

❖ Règle de Calcul de la Complexité :

1) Pour calculer la complexité grande O d'un algorithme il faut compter le nombre d'opérations de base qu'il effectue comme :
   - Opération arithmétique ou logique (+, -, *, /, et ,ou etc…)
   - Opération d'affectation (X←10)
   - Vérification d'une condition (x > 0)

- Opération d'entrée/Sortie (Ecrire ou Lire)

➢ La complexité de chaque opération de base est constante ou O(1)

2) La complexité d'une boucle est la complexité du bloc interne dans la boucle multipliée par le nombre de fois que le bloc interne est répété.

Exemple :

```
Pour i ← 1 a n pas 1 faire
    Ecrire (''Entrer un nombre a :'') ← Ecriture : O(1)
    Lire(a) ← Lecture : O(1)
    Ecrire ('' i * a :*, a * 1) ← Ecriture : O(1)
Pour
```

Affectation : O(1)
Opération arithmétique : O(1)

La complexité ci-dessus est : O(5n)

3) La complexité de la structure Si/Sinon correspond a la complexité de la condition (O(1)) plus la complexité la plus grande entre ''alors '' et ''Sinon''

Exemple :

```
Si n < 5 alors
    Ecrire (''hello word !'')
Sinon
    Pour i ← 1 a n pas 1 faire
        Ecrire (''hello word'')
    Fin pour
Fin si
```

vérification condition : O(1)
Ecriture : O(1)
Affectation : O(1)
Ecriture : O(1)

La complexité de l'exemple ci-dessous est : O(1) + Max(01), O(2n) = O(2n + 1)

4) La complexité d'une séquence de deux blocs d'instructions est égale à la plus grande des complexités des deux blocs.
5) La complexité d'un algorithme est un calcul de ses performances asymptotiques (données très volumineux) dans le pire des cas, les constantes multiplicatives sont remplacées par 1.
6) Les constantes additives sont annulées.
7) Le terme le plus élevé est conserve.